

A
MAJOR PROJECT REPORT
On
“ODIA ALPHA NUMERIC SIGN LANGUAGE DETECTION”

Submitted by
Manoranjan Moharana (2101110055)
Swadhin Ranjan Patra (2101110077)
Aswini Kumar Lenka (2101110035)
Adwaita Naraya Behera (2101110023)

Under The Esteemed Guidance of
Dr. Basanta Kumar Swain
(Asst. Professor)
&
Dr. Gopal Behera
(Asst. Professor)

In partial fulfilment of the award of the degree of
BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING



GOVERNMENT COLLEGE OF ENGINEERING
KALAHANDI, BHAWANIPATNA

May 2025

CERTIFICATE

This is to certify that this report “**ODIA ALPHA NUMERIC SIGN LANGUAGE DETECTION**” is the Bonafide work of “**Manoranjana Moharana (2101110055), Aswini Kumar Lenka (2101110035), Adwaita Narayan Behera (2101110023), and Swadhin Ranjan Patra (2101110077)**” who carried out the project work under my supervision. Certified further that the best of our knowledge the work reported herein does not form part of any other thesis or dissertation based on which degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. BASANTA KUMAR SWAIN
(SUPERVISOR)

Dr. GOPAL BEHERA
(SUPERVISOR)

EXTERNAL EXAMINER

HEAD OF DEPARTMENT
(CSE)

DECLARATION

We hereby affirm that the project work presented in this report, titled “**Odia Alpha Numeric Sign Language Detection**,” submitted to the **Department of Computer Science and Engineering** under the supervisions of Dr. Basant Kumar Swain and Dr. Gopal Behera, is our original work and fulfils the requirements for the Bachelor of Technology degree in Computer Science and Engineering. We certify that the work is free from any form of plagiarism, has not been submitted elsewhere for any other degree, and upholds the principles of academic honesty and integrity. Furthermore, we confirm that no idea, data, or source has been misrepresented, fabricated, or falsified in this submission.

MANORANJAN MOHARANA
(Regd. No.- 2101110055)

ASWINI KUMAR LENKA
(Regd. No.- 2101110035)

ADWAITA NARAYAN BEHERA
(Regd. No.- 2101110023)

SWADHIN RANJAN PATRA
(Regd. No.- 2101110077)

ACKNOWLEDGEMENT

The satisfaction of completing this report would be incomplete without mentioning the people who made it possible. Without their constant guidance and encouragement, our efforts would have been in vain. We consider ourselves privileged to express our gratitude and respect toward all those who guided us through the completion of this project. We convey our heartfelt thanks to our guide and supervisor, **Dr. Basanta Kumar Swain and Dr. Gopal Behera**, for providing encouragement, constant support, and guidance, which were of immense help in completing this project successfully.

We are also very grateful to **Dr. Gopal Behera**, Head of the Department of Computer Science and Engineering, for extending the support and encouragement necessary for the completion of this project.

Furthermore, we express our sincere gratitude to **Prof. (Dr.) Subhranshu Sekhar Dash**, Principal, Government College of Engineering, Kalahandi, Bhawanipatna, for providing us with a congenial environment to work in.

MANORANJAN MOHARANA
(Regd. No.- 2101110055)

ASWINI KUMAR LENKA
(Regd. No.- 2101110035)

ADWAITA NARAYAN BEHERA
(Regd. No.- 2101110023)

SWADHIN RANJAN PATRA
(Regd. No.- 2101110077)

ABSTRACT

Sign language is one of the most important tools for the specially-abled community to communicate with the world. Over the years, various approaches have been developed to support this need, including different forms of sign languages such as ASL (American Sign Language) and ISL (Indian Sign Language), which often vary by region and language. However, the absence of a standardized sign language system for the Odia language poses a significant challenge for specially-abled individuals within the Odia-speaking community, making it difficult for them to learn and express Odia alphabets and numbers. To address this, a custom image dataset was developed comprising static hand gesture images representing Odia characters and numbers. These images were captured from multiple individuals under various lighting and background conditions to ensure diversity, and each was manually labeled to ensure high annotation accuracy. In the preprocessing stage, we applied techniques such as image resizing and normalization to prepare the images for model input. Background removal was employed to eliminate irrelevant noise and isolate the hand gesture region, while image filters were applied to enhance important visual features such as edges and contours. Additionally, jitter-based augmentations including random rotation, zoom, translation, scaling, and brightness variation were implemented to artificially expand the dataset and simulate real-world inconsistencies in gesture presentation. For feature extraction, we utilized three powerful pre-trained Convolutional Neural Network (CNN) models: ResNet50 and EfficientNetB0, all initially trained on the ImageNet dataset. Rather than using these models for end-to-end classification, we removed their final layers and extracted high-level feature vectors from their intermediate outputs. These extracted features served as input to a Support Vector Machine (SVM) classifier using a radial basis function (RBF) kernel, which performed the final classification of hand gestures into their respective Odia characters. This hybrid approach leverages the deep representational power of CNNs with the robust and efficient classification capability of traditional machine learning. Model evaluation was conducted using metrics such as accuracy, precision, recall, and F1-score, and stratified train-test splits ensured balanced representation of all classes during evaluation. Among the models, EfficientNetB0 provided the most discriminative features, leading to the highest overall accuracy when coupled with the SVM. The research demonstrates a practical and efficient method for regional sign language recognition by combining deep CNN-based feature extraction with classical SVM classification.

Keyword: Sign Language, Odia Sign Language, EfficientNet-B0, Support Vector Machine (SVM), Machine Learning, Transfer Learning.

TABLE OF CONTENTS

DECLARATION	i
ACKNOWLEDGEMENT	ii
ABSTRACT	iii
CHAPTER 1.....	1
INTRODUCTION.....	1
1.1. Background of Sign Language Detection	1
1.2. Motivation.....	4
1.3. Outlines	4
CHAPTER 2.....	6
LITERATURE REVIEW	6
CHAPTER 3.....	8
PROBLEM FORMULATION AND PROPOSED MODEL ARCHITECTURE	8
3.1. Problem Statement	8
3.2. Proposed Architecture.....	8
CHAPTER 4.....	16
IMPLEMENTATION AND RESULT ANALYSIS	16
4.1. Dataset.....	16
4.2. Training Analysis.....	19
4.3. Result Comparison and Analysis	19
CHAPTER 5.....	24
CONCLUSION AND FUTURE SCOPE	24
5.1 Conclusion.	24
5.2 Future Scope	24
REFERENCES	25

LIST OF IMAGES

Images	Page No.
Fig 3.1: Proposed architecture of EficientNetB0+ SVM	8
Fig 3.2: Decision boundaries for multiclass classification in SVM	12
Fig 3.3: Kernel Trick	13
Fig 4.1: Illustration of background removal	18
Fig 4.2: Samples of augmented images	18
Fig 4.3: Distribution of images in training dataset	18
Fig 4.4: Distribution of images in test dataset	19

LIST OF TABLES

Tables	Page No.
Table 1: Split of Dataset	17
Table 2: Training Observations	19
Table 3: ASL result Comparisons with standard sources	22
Table 4: OSL data comparison	23

CHAPTER 1

INTRODUCTION

Communication is the cornerstone of human interaction, enabling individuals to convey thoughts, emotions, and information. Among the diverse modes of communication, speech remains the most dominant. However, for individuals with hearing or speech impairments, speech may not be a viable option. In such cases, alternative modes of communication like sign language become vital. Sign language is a visual-manual communication modality that employs a combination of hand gestures, facial expressions, body movements, and spatial orientation to convey meaning. It serves as a critical communication bridge for the deaf and mute communities, empowering them to interact with the world around them. Sign language can be divided into two main categories: static and dynamic. Static signs are steady hand and face gestures, while dynamic signs are divided into isolated and continuous signs. Isolated signs are hand gestures and facial expressions for a single word.

1.1. Background of Sign Language Detection

Historically, the use of hand gestures as a means of communication can be traced back to ancient civilizations. There is evidence suggesting that in ancient Greece, gestures were employed by individuals who were deaf to communicate. The modern development of sign language as a structured form of communication began in the 17th century, with the emergence of formal systems in Europe. One of the earliest records of structured sign language education dates back to the efforts of Juan Pablo Bonet in Spain, who published the first book on sign language for the deaf in 1620 [1]. In the 18th century, Charles-Michel de l'Épée, a French educator, established the first free public school for the deaf, formalizing the teaching of French Sign Language and laying the foundation for sign languages worldwide [2]. American Sign Language (ASL) emerged in the early 19th century, heavily influenced by French Sign Language (LSF) through the collaboration of Thomas Hopkins Gallaudet and Laurent Clerc. ASL has since evolved into a rich and complex language with its own grammar, syntax, and regional variations. It is widely used in the United States and parts of Canada, and it plays a significant role in the cultural identity of the Deaf community in these regions [3]. In India, sign language development followed a different trajectory. Indian Sign Language (ISL), also known as Indo-Pakistani Sign Language (IPSL), is the primary sign language used by the deaf community in the Indian subcontinent. Despite its extensive use by over 15 million individuals, ISL lacked formal recognition and structured educational support for many years. It was only in the early 2000s that systematic efforts were made to promote and teach ISL. The Ali Yavar Jung National Institute for the Hearing Handicapped (AYJNHH) in Mumbai was one of the first institutions to offer a diploma course in ISL, aiming to professionalize sign language interpretation and raise awareness [4]. In 2005, the Indian government began integrating ISL into educational curricula, with the National Council of Educational Research and Training (NCERT) publishing content on sign language for school textbooks [5].

The diversity of sign languages worldwide is immense, with over 150 recognized sign languages, each shaped by cultural, geographic, and historical influences. Despite this richness, a major challenge persists: the communication gap between sign language users and non-signers. Most individuals outside the Deaf community lack the ability to understand sign language, which results in social exclusion, miscommunication, and limited opportunities for the hearing-impaired. This communication barrier underscores the urgent need for technological solutions that can bridge the gap and facilitate inclusive interaction. With the rapid advancement in technology, particularly in the field of artificial intelligence (AI) and computer vision, significant strides have been made in developing systems that can recognize and translate sign language into spoken or written language. These systems typically fall into two categories: glove-based and vision-based technologies. Glove-based systems use sensor-equipped gloves to capture hand movements and gestures. While these systems offer high accuracy, they are often expensive, cumbersome, and require users to wear specialized hardware, limiting their practicality for everyday use [6]. In contrast, vision-based systems utilize standard cameras and computer vision algorithms to interpret gestures without the need for wearable devices. These systems are more user-friendly and accessible, making them a preferable option for real-world applications. However, they face challenges such as sensitivity to lighting conditions, background noise, and the complexity of accurately recognizing dynamic gestures [7].

Recent research has increasingly focused on the use of deep learning techniques for sign language recognition. Deep learning, a subset of machine learning, mimics the neural structure of the human brain to identify patterns and features in large datasets. Convolutional Neural Networks (CNNs) have been particularly effective in image classification tasks, making them a popular choice for gesture and sign recognition systems. CNNs automatically extract spatial hierarchies of features from input images, eliminating the need for manual feature engineering and significantly improving the accuracy of recognition. In the context of sign language recognition, CNNs can be trained on large datasets of hand gestures to identify and classify signs. Transfer learning, another powerful technique in deep learning, allows researchers to leverage pre-trained models such as VGG16, ResNet50, InceptionV3, and MobileNetV2. These models, trained on massive image datasets like ImageNet, can be fine-tuned to recognize sign language gestures with relatively smaller datasets. This approach accelerates the development process and enhances performance, even in resource-constrained environments [8].

Numerous studies have explored sign language recognition using deep learning. For instance, models have been developed to recognize ASL alphabets and numbers, translating finger-spelled gestures into text. These models have demonstrated high accuracy and robustness in controlled environments. Similarly, research on ISL has focused on recognizing static hand gestures representing common words or alphabets. Given the lack of standardized datasets for ISL, many researchers have created custom datasets, capturing images or videos of hand gestures under varied lighting and backgrounds to improve generalization [9]. The classification of sign language gestures is generally divided into static and dynamic categories. Static signs involve fixed hand shapes and positions, while dynamic signs include

movement over time, often forming words or sentences. Dynamic signs are further categorized into isolated and continuous signs. Isolated signs represent single words, whereas continuous signs form complete sentences, requiring the system to interpret a sequence of gestures [8]. Building models that can handle continuous signs remains a complex challenge due to the temporal dimension involved. To address this, researchers have started integrating Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks with CNNs. RNNs and LSTMs are designed to process sequential data, making them suitable for interpreting dynamic signs. When combined with CNNs for feature extraction, these architectures can learn temporal patterns in video data, enabling more accurate recognition of continuous gestures [10]. In addition to recognition, some systems aim to generate spoken language or text from recognized signs, effectively acting as real-time translators. Such systems have the potential to revolutionize communication for the Deaf community, particularly in public places like hospitals, banks, and educational institutions. Moreover, these technologies can be integrated into mobile applications, wearable devices, or embedded systems for broader accessibility [11]. Despite the progress, several challenges remain. Variability in signing styles, occlusion of hands, changes in background, and limited availability of labelled datasets pose significant hurdles. Ethical considerations such as privacy, inclusivity, and cultural sensitivity must also be addressed. Furthermore, ensuring that these technologies are affordable and scalable is crucial for widespread adoption [12].

Sign language recognition is a multidisciplinary field that intersects linguistics, computer vision, and machine learning. The integration of deep learning techniques has opened new avenues for developing robust and accurate systems capable of bridging the communication gap between sign language users and the wider society. This work explores and implements a vision-based sign language recognition system, focusing on Indian Sign Language using deep learning methodologies [13]. By leveraging CNNs and transfer learning, the goal is to build a system that can accurately recognize hand gestures and translate them into meaningful text, contributing to a more inclusive and connected world. Odia Sign Language remains underrepresented in the field of gesture recognition. This work takes a significant step toward bridging that gap by exploring the application of deep learning techniques for recognizing Odia hand gestures. We tried to solve this problem using deep learning techniques such as convolutional neural networks (CNNs) and transfer learning. The aim is to create an effective and accurate vision-based recognition system tailored to the unique characteristics of Odia gestures. This initiative not only contributes to the advancement of sign language technology but also promotes inclusivity for the Odia-speaking deaf and hard-of-hearing community. The dataset was carefully curated to include a diverse range of Odia hand gestures representing the alphabets and numbers. Preprocessing techniques were applied to enhance image quality and reduce noise, improving model performance. Experiments were conducted using multiple CNN architectures to evaluate accuracy, precision, and recall. Transfer learning helped optimize performance even with a relatively limited dataset. The results demonstrate the feasibility of applying modern deep learning approaches to regional sign languages. Ultimately, this research lays the groundwork for future studies and real-time applications supporting Odia Sign Language users.

1.2. Motivation

During our research into various sign languages and their technological applications, we observed that most existing work has been extensively concentrated on standardized sign languages such as American Sign Language (ASL) and Indian Sign Language (ISL). These languages benefit from large, annotated datasets, consistent gesture definitions, and a strong presence in academic and industry research. However, we found no significant work has been done for Odia Sign Language (OSL), a regional sign language used by the Deaf and Hard of Hearing (DHH) community in Odisha, India. This lack of attention creates a significant accessibility gap for Odia-speaking individuals, as they are left without technological support or inclusion in current assistive communication tools. Given that Odia is spoken by over 35 million people and is the official language of Odisha, this oversight is both socially and technologically critical. The development of an Odia Sign Language detection system using deep learning techniques can address this issue by enabling real-time recognition and translation of OSL gestures. Deep learning, especially models such as Convolutional Neural Networks (CNNs), has proven highly effective in visual recognition tasks and can handle the spatial complexity involved in sign language gestures. By leveraging these models, we can design a robust system that recognizes Odia sign gestures and translates them into text or speech, fostering communication between DHH individuals and the hearing population. Furthermore, such research contributes to regional language preservation, enhances digital inclusion, and supports government efforts toward inclusive technology under initiatives like the Accessible India Campaign. Thus, there is a pressing need to fill this gap through dedicated research and development in Odia Sign Language detection using deep learning, ultimately empowering a largely underserved community.

We have been working on Odia Sign Language (OSL) and encountered several challenges during our research. One of the major issues is the absence of standardized signs, which makes it difficult to create consistent datasets and models. Additionally, there is no existing research or technological framework specifically developed for OSL, highlighting a significant gap in the field. This lack of resources poses a challenge but also presents an opportunity to contribute foundational work in this area.

The main contributions of our work are as follows:

- We developed a model EfficientNetB0 with Support Vector Machine (SVM) for classification.
- We collected an Odia Sign Language dataset and attempted to standardize it.
- We are conducting several experiments on our model to improve feature extraction and classification.
- We are using a real-world OSL dataset to train our model and achieve better results.
- Our proposed model achieved an accuracy of 99.69%.

1.3. Outlines

In Chapter 2, Literature Review, we review various research papers that have worked on different sign languages.

Chapter 3 presents the architecture of our model. It details the components and structure designed to recognize Odia Sign Language.

In Chapter 4 Result Analysis, we compare the performance of our model with various other models. We evaluate and analyze the results based on key metrics to assess the effectiveness of our approach.

In the last Chapter, Conclusion and Future Scope, we discuss the key findings and conclusions of our project. We also highlight potential improvements and future directions for further research and development in Odia Sign Language recognition.

CHAPTER 2

LITERATURE REVIEW

Numerous studies have explored sign language and hand gesture recognition using machine learning and deep learning techniques. Vision-based approaches, such as CNNs and RNNs, have been widely used for feature extraction and gesture classification. While many models show high accuracy under controlled conditions, they often struggle with variations in background, lighting, and hand orientation. Some research focuses on static gestures (alphabets and numbers), while others address dynamic sequences for full sentence recognition. Despite significant progress, achieving real-time performance and generalization across users remains a key challenge. These limitations highlight the need for more robust and scalable models.

Rokade et al. [14] developed a vision-based system for recognizing static gestures in Indian Sign Language (ISL). The method uses image preprocessing techniques like skin color segmentation, binary conversion, and Euclidean distance transformation. Features are extracted using central moments, Hu moments, and Fourier descriptors. Two classifiers—Artificial Neural Networks (ANN) and Support Vector Machines (SVM)—were used for recognition. Testing on 17 ISL signs showed ANN outperformed SVM with a maximum accuracy of 94.37% using 13 features. The system effectively bridges communication barriers for the deaf and mute, using only standard cameras without the need for wearable sensors. Taskiran et al. [15] developed a real-time American Sign Language (ASL) recognition system using a CNN-based model trained on a 2011 Massey University dataset. The model architecture includes convolution, pooling, and dense layers trained on 900 grayscale images of 36 ASL characters (A–Z, 0–9), resized to 28×28 pixels. The system integrates CNN classification with a convex hull-based hand detection algorithm using skin color segmentation in YCbCr space. It achieved 100% test accuracy during training and 98.05% accuracy in real-time tests. Designed for efficiency, the system runs within ~25ms per frame, enabling smooth real-time application. Future improvements may include expanding dataset size and refining recognition under variable conditions. Ali et al. [16] conducted a comparative analysis of YOLOv3, YOLOv4, and YOLOv5 for sign language detection using a dataset of 2000 images. The study evaluates each model based on precision, recall, and mAP@0.5 over 100 training epochs. YOLOv3 uses the Darknet-53 backbone, YOLOv4 enhances this with CSPDarknet53 and PANet, while YOLOv5 further improves performance using a Focus structure, SPP block, and GIoU-loss. The results show YOLOv5 outperformed others with precision (0.72), recall (0.81), and mAP@0.5 (0.87). YOLOv4 followed closely, while YOLOv3 had the fastest inference but lower accuracy. The study concludes YOLOv5 is best suited for sign detection tasks requiring high accuracy, though YOLOv3 may still be preferable in time-critical applications.

Kothadiya et al. [17] proposed a deep learning-based system for recognizing isolated Indian Sign Language (ISL) gestures from video sequences. Using a custom IISL2020 dataset and a hybrid LSTM-GRU architecture, the model achieved a recognition accuracy of 97% for 11 commonly used ISL signs. Feature extraction was performed using InceptionResNetV2, and

various RNN configurations (LSTM, GRU, and their combinations) were tested. Among these, the LSTM-GRU model outperformed others, with consistent results across other datasets like AUTSL and GSL, demonstrating robust performance without the need for controlled environments or sensor-based inputs. Mohsin et al. [18] explored the recognition of American Sign Language (ASL) letters and numbers using deep learning and transfer learning approaches. Five models—CNN, VGG16, ResNet50, MobileNetV2, and InceptionV3—were trained and evaluated on a dataset derived from the MNIST ASL database. Among these, InceptionV3 achieved the highest accuracy (96%), recall (97%), and F1 score (96%). The study demonstrated the strength of transfer learning, especially in situations with limited labeled ASL data. While InceptionV3 stood out, models like VGG16 (95% accuracy) and MobileNetV2 (92%) also showed promising results, offering flexibility based on hardware and application requirements. Saini et al. [19] investigated Indian Sign Language (ISL) recognition using various deep learning and transfer learning models. A custom dataset of 10,400 images across 26 ISL alphabet classes was created under varied lighting and background conditions. Seven models—Basic CNN, VGG16, VGG19, ResNet-50, DenseNet-201, MobileNet V2, Inception V3, and Xception—were evaluated. Among them, ResNet-50 achieved the highest performance with 99.26% test accuracy and a 99.34% F1-score, while Inception V3 performed the worst (70.54% accuracy). The study highlighted the importance of dataset quality and deep model architecture in achieving robust ISL recognition.

Orovwode et al. [20] developed a real-time American Sign Language (ASL) recognition system using a vision-based approach and a Convolutional Neural Network (CNN). A dataset of 44,654 static ASL alphabet images (excluding dynamic signs J and Z) was collected via webcam using the HandDetector module. Images were preprocessed and split into training, validation, and test sets. The CNN model, with three convolutional layers and a Softmax output layer, was trained using the Adam optimizer. It achieved 99.86% training accuracy, 99.94% validation accuracy, and 94.68% test accuracy. While effective in static gesture recognition, the study suggests future work to address dynamic gestures and broader sign language diversity. Rambhia et al. [21] proposed a real-time Indian Sign Language (ISL) recognition system using Deep LSTM networks and hand key point detection. The system utilizes Google's MediaPipe to extract 21 key points per hand and feeds a sequence of these (from 30 video frames) into a stacked LSTM model to classify signs. The model was trained on 700 annotated videos with 18,663 sentence-level frames and 1,036 word-level images. It achieved 91.5% accuracy in recognizing both words and phrases in real-time. The approach outperformed traditional bounding-box-based models, benefiting from precise pose estimation and sequence learning. YOLOv3 was also used for gesture detection with precision up to 93.6%. The study highlights vision-based methods over sensor-based systems for scalable and cost-effective deployment. Misal et al. [22] presented a real-time sign language detection system using a YOLO-based model trained on real-world Indian Sign Language (ISL) video data. The system architecture includes data acquisition, preprocessing, annotation, and training with a pre-trained YOLO model, optimized for speed and practicality. The model was trained on 5,000 annotated frames representing 7 commonly used ISL signs and achieved 87.5% accuracy, with a detection speed of 15 FPS. The study highlights YOLO's efficiency and suitability for deployment in real-time assistive applications.

CHAPTER 3

PROBLEM FORMULATION AND PROPOSED MODEL ARCHITECTURE

The following section discusses the problem statement concerning the detection and recognition of Odia sign language.

3.1. Problem Statement

Although there is numerous research have been conducted on the detection of standard sign languages such as American Sign Language (ASL), currently there is no formal systems or significant work available for Odia sign language. In this project we explored various machine learning and deep learning techniques to develop an effective solution for the detection of Odia sign language.

3.2. Proposed Architecture

In this section, we present the proposed architecture and outline the workflow steps followed throughout the project. The complete workflow of the architecture is shown in figure 3.1.

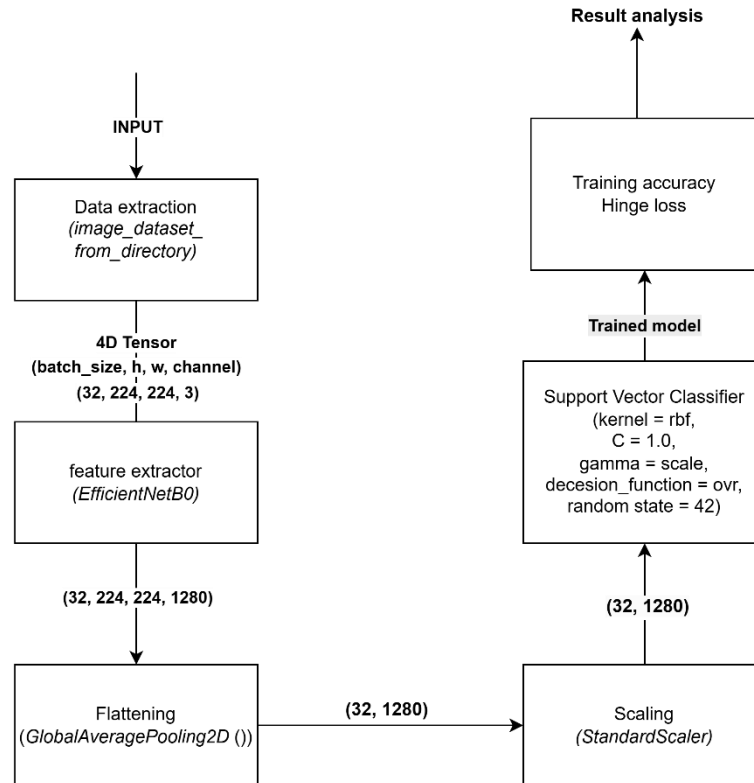


Figure 3.1 Proposed architecture of EfficientNetB0+ SVM

3.3. Methodology

In this section, we discussed the complete methodology we followed for training the model, describing step-by-step how the model gradually learned the features of the

images.

3.3.1.Feature Extraction

We used the pretrained model EfficientNetB0 for feature extraction in the Odia Sign Language Detection system. The models are based on Convolutional Neural Network (CNN) architectures and were selected for their proven performance in image-related tasks. For input handling, we used *image_dataset_from_directory* from the *tf.keras.preprocessing* module to process the hierarchical dataset. This function automatically loads the image data and assigns numerical labels (starting from 0) based on the alphabetical order of the subdirectories. After preprocessing, the function returns a 4-dimensional image tensor with the shape (*batch_size*, *height*, *width*, *channels*) and a corresponding label tensor of shape (*batch_size*,). For our project, we mathematically represented the input tensors as

$$X_{in}(batch, h, w, c) = (32, 224, 224, 3) \quad (1)$$

As shown in equation 1, we resized the images to a fixed size of ‘224×224’ pixels, which is a standard input size for EfficientNetB0, and batching the data (with a batch size of 32 in our setup). In equation 1, c stands for the number of channels; at input we have 3 channels of RGB [23].

We applied the input image tensor X_{in} to the EfficientNetB0 model for feature extraction. In EfficientNetB0, the tensors applied to the MBConv layers which are analogous to the convolution layers of traditional CNN architecture. MBConv is stand for Mobile Inverted Bottleneck Convolution layers. They are based on the number of channels, where channels refer to the feature maps or grids of an image at any point. The number of channels indicates the number of feature maps of the image. More channels help the model learn more detailed features but also increase computational costs. This situation is referred to as a bottleneck. A bottleneck occurs when the model's capacity is limited due to insufficient resources, much like the narrow neck of a bottle. It's difficult for things to pass through a narrow neck, but if they do, a lot of space becomes available. This analogy applies to the model's capacity—if it can overcome the bottleneck, the model will yield better results [24]. MBConv layers are consists of multiple layers of convolution. The first layer in MBConv is expansion layer. The expansion step uses a 1x1 pointwise convolution to increase the number of channels. Pointwise convolution refers to convolving the image patch with a filter of size 1x1. The resulting number of image channels depends on the number of filters. The number of channels in the input tensor, multiplied by the expansion ratio, gives the total number of filters, which is equal to the resulting number of channels in the output tensor. We feed the expansion layer with the input tensor X_{in} as shown in equation 2.

$$X_e(i, j, k) = \sum_{c=1}^C W_{k,c} \cdot X_{in}(i, j, c) + B_k \quad (2)$$

Where, $X_{in}(i, j, c)$ is the input value at position (i, j) of channel c, $W_{k,c}$ is weight for the k-th output channel and c-th input channel, B_k is bias for k-th output channel, $X_e(i, j, c)$ is the output value at position (i, j) in channel k, C is the number of input channel and k are the indices of output channel. This operation computes a weighted sum across input channels for each spatial location (i, j), producing an output with new

channel dimensions.

Then we applied the output of expansion layer $X_e(i, j, c)$, as input to the depth wise convolution layer. This layer applies convolution to each filter on the channels independently according to the mathematical equation:

$$X_d(i, j, k) = \sum_{m=1}^k \sum_{n=1}^k W_k(m, n) \cdot X_e(i + m, j + n, k) + B_k \quad (3)$$

Where, $X_e(i, j, k)$ is value at position (i, j) of channel k i.e. the output of previous expansion step and input to the depth wise convolution, $W_k(m, n)$ is the weight at position (m, n) for the k-th channel, B_k is the bias of the k-th output channel, $X_d(i, j, k)$ is output value at position (i, j) for k-th output channel and z is filter size index. In this layer There is no combination between the weighted convolution values of the channels, which reduces computations and speeds up the process [25].

After this, we used the output X_d of the previous depth wise layer to the next layer, the squeeze and expansion layers. In the squeezing step, the feature map of each channel is condensed to a single value using global average pooling (GAP), which calculates the average value of the entire feature map.

$$s_k = \frac{1}{h \times w} \sum_{i=1}^h \sum_{j=1}^w X_d(i, j, k) \quad (4)$$

Where, h and w are height and width of the input feature map respectively, s_k is the output value of c-th channel after pooling and $s = [s_1, s_2, \dots, s_c]$ is the vector that stores the global average for each channel. This process as shown in equation 4, reduces the spatial dimensions while keeping the number of channels the same. The condensed value helps in learning the weight values. By summarizing each feature map into a single value, GAP eliminates the need for fully connected layers that typically follow convolutional layers. This reduction in layers lowers the number of parameters in the model, leading to faster training times and a reduced risk of overfitting. After squeezing, the squeezed vector s is passed through two fully connected layers (also known as dense layers) to learn a set of channel-wise weights that capture the importance of each channel, using the mathematics of equation 5.

$$e = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot s)) \quad (5)$$

Where, e are scalars that represent how much each channel should be emphasized or suppressed, W_1, W_2 are the weight matrices of fully connected layers, ReLU applied to the first fully connected layer and similarly Sigmoid applied to the second fully connected layer. These fully connected layers introduce non-linearity into the model, enabling it to learn complex dependencies between channels. The excitation step then captures the channel-wise dependencies. In traditional CNNs, all channels are treated equally, but some channels may carry more useful information than others. The excitation step learns the weights based on the importance of each channel, and these weights are applied to the feature maps, ensuring that only the most important features are retained. Then we applied the extracted expansion factor e, with the existing feature maps X_d gives the channel-wise attention to the original features. This adjusts the feature map by scaling the importance of each channel based on the excitation values.

$$X_{se} = e \cdot X_d \quad (6)$$

Now again the pointwise convolution is done to project the feature maps to the original number of channels.

$$X_{out}(i, j, c) = \sum_{k=1}^C W_{k,c} \cdot X_{se}(i, j, k) + B_c \quad (7)$$

Where, $W_{k,c}$ is the weight associated with the c -th input channel and the k -th output channel in the 1×1 convolution, B_c is the bias term for the c -th output channel, C is the number of input channels in X_{se} , $X_{out}(i, j, c)$ is the output feature map value at position (i, j) and the c -th channel and $X_{se}(i, j, k)$ is the input feature map value at position (i, j) and the k -th channel, which is the feature map after the Squeeze-Expansion block.

Then we applied the residual connection as per the equation 8. We add the input with the output tensors from Squeeze and Expansion layers, which helps in smooth feature propagation and addresses the vanishing gradient problem by preserving minute details that might otherwise be lost during feature extraction [24].

$$Y(i, j, c) = X_{in}(i, j, c) + X_{out}(i, j, c) \quad (8)$$

Equation 9 described the Swish activation function, which we used during the feature extraction process in EfficientNetB0. The Swish function is essentially the product of the original value and the sigmoid of that value. Mathematically, it is represented as:

$$Swish(x) = x \times \sigma(x) \quad (9)$$

Where $\sigma(x)$ is sigmoid function. Swish is more efficient because it does not completely cut off all negative values. Instead, it smoothly allows small negative values to pass through. If the value of x is large and positive, $\sigma(x)$ is approximately 1, so the Swish output is close to x , giving it high weight. However, if x is small or negative, $\sigma(x)$ becomes small, and the Swish output is suppressed but not entirely removed, just given less importance. This allows more information to flow through the network, resulting in better performance [26]. The output from the MBConv layers is passed through an activation layer as

$$Z(i, j, c) = Swish(Y(i, j, c)) \quad (10)$$

The extracted feature maps are then flattened using the GlobalAveragePooling2D (GAP) function. This function reduces each feature map (a 2D grid of values) into a single number by taking the average of all values in that map. Mathematically, for a feature map of size, the output is:

$$x_c = GAP(c) = \frac{1}{h \times w} \sum_{i=1}^h \sum_{j=1}^w Z(i, j, c) \quad (11)$$

Where, h and w are the height and width of the feature map, $Z(i, j, c)$ is the output of the final feature extractions that need to flatten for use, c stands for channel index, $GAP(c) = x_c$ is a scalar value of dimensions $(C,)$ and C is the total no of channels in the feature map stack. Flattening by the GAP function simplifies the data while retaining the core information from each feature map. Now, each channel has only one value, which is the average pooled value, instead of the entire feature map matrix. These reduced and flattened features were then saved in *.npz* files with the shape *(image_number, number_of_features)* for images and *(image_number,)* for labels, for future use [27].

3.3.2.Feature Scaling

After feature extraction and flattening, we applied Standard Scaling to the extracted features before using them for classification. Standard Scaling is an important preprocessing step in machine learning, especially when the features have different scales. It ensures that each feature contributes equally to the model by centering the data around zero mean and scaling it to unit variance. This helps improve the performance and convergence speed of many algorithms, particularly those based on distance measures or gradient descent. The process involves calculating the mean and standard deviation of each feature across all samples in the training set and then transforming each feature using the formula:

$$x' = \frac{(x_c - \mu)}{\sigma} \quad (12)$$

Where, x_c is the scaler value from each channel that pooled by GAP function, μ is notation for mean of the feature, σ is for standard deviation of the feature and x' is standardized (scaled) feature values. X' is the set of all scaled feature vectors x' calculated by equation 12. This transformation ensures that each feature has a mean of 0 and a standard deviation of 1. It's important to note that the scaler is fitted on the training data only to prevent data leakage, and the same parameters (μ and σ) are used to scale the validation and test data [28].

3.3.3.Model Training

After final feature extraction and scaling, we then used the resultant scaled features for model training of our classifier model. In the SVC-based architecture, we used Support Vector Classifier (SVC) to classify images based on features extracted by pre-trained models. To handle non-linear data distributions, the RBF (Radial Basis Function) kernel was applied. For multi-class classification, the One-vs-Rest (OvR) decision function strategy was employed, allowing the model to effectively distinguish between multiple classes, as shown in Figure 3.2.

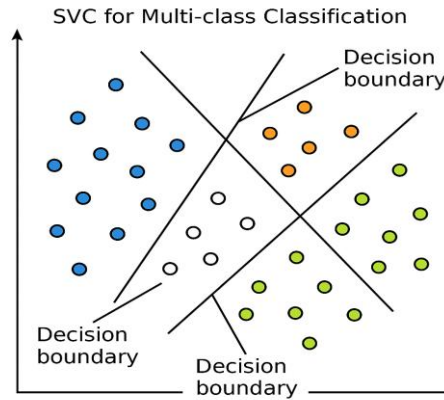


Figure 3.2 Decision boundaries for multiclass classification

For model training, we used the *Support Vector Classifier (SVC)* to classify images based on the features extracted from pre-trained models. SVC is a variant of Support Vector Machines (SVM), which includes other forms like *LinearSVC* i.e. faster but less accurate for complex, non-linear data and *SGDClassifier*, which is efficient for extremely large datasets but may converge less accurately. Among these, SVC with a non-linear kernel was best suited for our moderately sized dataset with complex class boundaries.

To handle this non-linearity, we used the *Radial Basis Function (RBF)* kernel. Kernels are used in SVM to allow classification even when data is not linearly separable in its original form. This is done through the *kernel trick*, which mathematically transforms the feature space to a higher dimension where the data can be linearly separated. For instance, imagine data that can't be separated with a straight boundary i.e. mapping it into a higher space allows SVM to find a plane or boundary that separates it cleanly, and the kernel trick does this without explicitly computing the transformation. Figure 3.3 describes a scenario of kernel trick for 2 dimensional nonlinear features. The RBF kernel is particularly effective in this, as it measures similarity between data points and allows drawing flexible decision boundaries.

The mathematical formula for the RBF kernel is:

$$K(x_a, x_b) = e^{-\gamma \cdot \|x_a - x_b\|^2} \quad (13)$$

Here x_a and x_b represent two different data points (e.g., two images) in the dataset and where γ (gamma) controls how much influence each training example has higher values mean more localized influence. Each vector contains the numerical features extracted from data samples, like the 2048 features from ResNet50 or 1280 from EfficientNetB0.

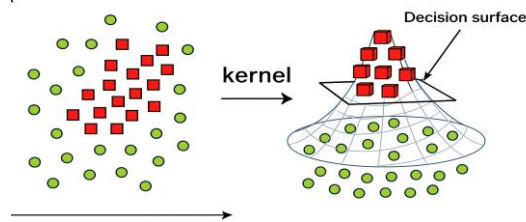


Figure 3.3: Example of kernel trick

$\|x_a - x_b\|^2$ is the *Euclidean norm (or L2 norm)* of the vector $x - x'$. It measures the distance between two points x and x' in the feature space. Norm is a way to measure the length or size of vectors [29].

$$\|x_a - x_b\| = \sqrt{(x_{a1} - x_{b1})^2 + (x_{a2} - x_{b2})^2 + \dots + (x_{an} - x_{bn})^2} \quad (14)$$

To support multi-class classification, we used the One-vs-Rest (OvR) strategy via `decision_function_shape = 'ovr'`, which trains one binary classifier per class, each

distinguishing one class from all others. This results in multiple decision boundaries in higher-dimensional space, enabling robust multi-class predictions.

In this SVM configuration, *kernel='rbf'* means the model uses the Radial Basis Function kernel, which is good for handling complex, non-linear data patterns. The parameter *C=1.0* balances between correctly classifying training examples and keeping the model simple i.e. lower values of C allow some mistakes but aim for a wider margin between classes. *gamma='scale'* automatically sets the gamma value based on the number and spread of features, which helps the model decide how far the influence of a single data point should reach. The *random_state=42* makes sure the model gives the same results every time you run it, which is helpful for reproducibility. *decision_function_shape='ovr'* tells the model to use a One-vs-Rest strategy for handling multiple classes, this means it builds one classifier per class [30].

The SVC (Support Vector Classification) in scikit-learn does not directly expose a loss function, but internally it uses the *hinge loss* as part of a quadratic optimization process. This loss is defined as

$$L(y, f(x)) = \max(0, 1 - y \cdot f(x)) \quad (15)$$

The Hinge Loss measures the error between the true label y and the decision function value $f(x)$. If the decision function $f(x)$ correctly classifies the input with a margin of at least 1, the loss will be 0. Otherwise, the loss increases as the decision function value deviates from the correct classification. If $y \cdot f(x) \geq 1$ it is no loss i.e. the model classifies the sample correctly with sufficient confidence) and if $y \cdot f(x) < 1$ means A positive loss, which increases as $y \cdot f(x)$ moves further from 1. Thus, hinge loss provides insight into both correctness and confidence of the classifier [31].

We applied the scaled features x' from equation 12 in to the model. The equation 16 and 17 shows the mathematical form of the proposed SVM model.

$$f(x) = \sum_{i=1}^n \alpha_i y_i \cdot e^{-\gamma \cdot \|x_i - x\|^2} + b \quad (16)$$

Where, $x, x_i \in X'$, where X' is the set of standardized feature vectors from eq 12, x is Standardized feature vector of the current input (sample to classify), x_i is Standardized feature vector of the i -th support vector (from training set), α_i is weight assigned to the i -th support vector, y_i is true class label (+1 or -1) of the i -th support vector, γ is kernel coefficient, determine how loosely or tightly 2 points consider to be similar. It calculated automatically if $\gamma = 'scale'$, b is bias term learned during training, n is for number of support vector and $f(x)$ is the value that indicates similarity extent between input feature x and support vector x_i . The scaled features are used as inputs to the decision function (eq. 16) that calculates the similarity between 2 features, by a real valued score $f(x)$, called decision score. The predicted value is determined by sign of $f(x)$. the label predicted is +1 for positive $f(x)$ and -1 for the opposite.

Then the decision score is given input to the objective function of SVM (eq. 17), to recalibrate the training weights.

$$\text{Minimize } \left(\frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \text{hinge loss}(y_i, f(x_i)) \right) \quad (17)$$

$\frac{1}{2} \|w\|^2$ is the regularization term that helps to maximize the margin between classes. Minimizing this term encourages the model to find a decision boundary with the largest possible margin. w the weight vector that defines the decision boundary of the SVM model. $C \sum_{i=1}^n \text{hinge loss}(y_i, f(x_i))$, where C is the regularization parameter that controls the balance between margin size and classification error, calculate the hinge loss, i.e. calculate how much prediction of the classifier is wrong and penalize accordingly [32].

At last, model training is complete when the algorithm has found the optimal values for the weights (w) and bias (b), and the objective function has been minimized. This ensures that the model has learned the best decision boundary, balancing the margin size and classification accuracy effectively

Finally, we saved the trained model as a *.pkl* file using Python's pickle module for further result analysis and reusability without retraining.

CHAPTER 4

IMPLEMENTATION AND RESULT ANALYSIS

4.1. Dataset

The following describes the creation, collection, and implementation of datasets used in our report.

4.1.1. American Sign Languages (ASL) dataset

We used a publicly available ASL dataset from Kaggle [33], which contains 27000 images distributed across 27 classes—all 26 letters of the English alphabet and random images for blank. ASL is a standardized one-handed sign language that is widely accepted and used around the world. Since this dataset follows established sign language conventions, it served as a good reference point and helped us understand how standard systems represent letters and numbers.

4.1.2. Odia Sign Language Creation

As our project specifically focuses on Odia sign language, the main dataset we used was the one we created ourselves. Unlike English or ASL, there was no standardized dataset for Odia letters or numbers available before. This presented a significant challenge: we had to design a set of sign language symbols for the Odia script from scratch.

The process began with the careful creation of a comprehensive dataset tailored for Odia sign language recognition. We collected images representing each sign, ensuring variations in hand orientation, lighting conditions, and backgrounds to enhance the dataset's diversity and robustness. Each image was categorized and labeled according to the corresponding Odia number or alphabet it represented. To ensure consistency and quality, we maintained a standardized image resolution and format throughout the dataset. Once collected, the dataset was preprocessed and augmented to improve the model's generalization capabilities. It was then divided into training and testing subsets and implemented in the development of machine learning models for accurate sign recognition and classification.

Challenges Faced

Designing a sign language system for Odia was not straightforward. Some key challenges included:

- **More characters:** While English has only 26 letters, Odia has around 48 primary letters, including 12 vowels and numerous consonants.
- **Consonant ligatures:** Some letters in Odia are primarily used in consonant combinations (ligatures) and are not used independently.
- **Script complexity:** Like many Indian languages, Odia includes diacritics (matras), "phala", and "yuktakshara" (ligatures). These elements combine base letters to create new sounds, making it difficult to represent every possible

combination with a single sign.

- No standard mapping: Since there was no pre-existing standard for Odia signs, our main focus is to make signs clear, understandable and easy to remember.

Our Approach

Given the above challenges, we took a foundational and practical approach:

- We limited our initial scope to only base letters, excluding diacritics and complex ligatures.
- To reduce redundancy, we merged letters with similar sounds. For example:
 - ଇ and ଈ both represent the "i" sound, so we kept only ଇ.
 - ଓ and ଔ often sound similar, so we kept only ଓ.
 - Among ଶ, ଷ, and ସ, we kept only ସ.
 - Between ଲ and ଳ, we kept ଳ.
- Letters rarely used independently, like ଙ and ଞ, were excluded in this version.

We also focused on consistency and user understanding. For example, letters with similar sounds (like ଢ and ଡ) were given signs with similar visual structure, help the algorithm to minimize error in detection.

4.1.3.Data Collection and Preprocessing

We used the *VideoCapture()* function from the *OpenCV* library to collect images directly through our system's webcam. In total, we captured 4,800 images, unevenly distributed across 48 classes. To improve the clarity of the signs, we pre-processed the data by removing the backgrounds, which helped enhance the focus on hand gestures. After preprocessing, we split the dataset, where the larger portion was used for training and the rest for testing. Table 4.1 describe the splitting of data in 6:4 ratio where 2880 data images are used for training purposes and rest 1920 data images are used for testing purposes.

Table 4.1:Train Test Split

DATA	Total Dataset	Total Class	Split	Training dataset (without augmentation)	Testing dataset
OSL	4,800	48	6:4	2880	1920
ASL	27,000	27	9:1	24300	2700

To expand the training dataset and introduce variety, we applied data augmentation techniques using the '*Albumentations*' Python library (not in real-time). These techniques included shifting, scaling, rotation, noise addition, compression and many more [34]. Figure 4.1 shows the Background removal of ଳ, illustrating how

the character can be isolated for further processing or analysis.
The set of techniques that results in the augmentations can be shown mathematically

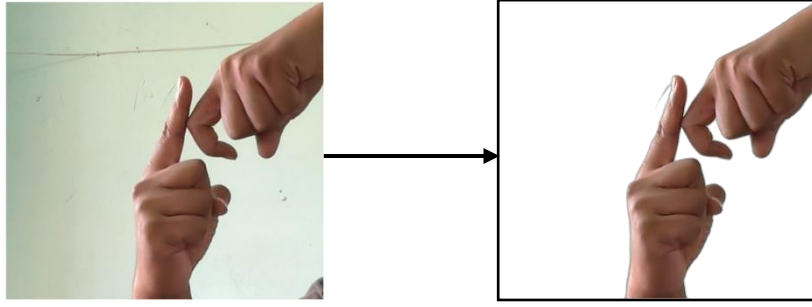


Figure 4.1 Background removal of $\mathcal{Q}$

as,

$$f_{aug}(x) = f_{griddropout}(f_{downscale}(f_{imagecompression}(f_{isonoise}(f_{gaussiannoise}(f_{rgbshift}(f_{gaussianblur}(f_{hsv}(f_{perspective}(f_{rotate}(f_{shiftscale}(x))))))))))) \quad (19)$$

Figure 4.2 shows the samples of augmented images of $\mathcal{Q}$, illustrating how various transformations are used to generate the augmented dataset.



Figure 4.2 Samples of augmented images of $\mathcal{Q}$

This augmentation process significantly increased the training dataset size, resulting in 28,800 images, with exactly 600 images per class across all 48 classes. Figures 4.3 and 4.4 shows the distributions of images across the classes.

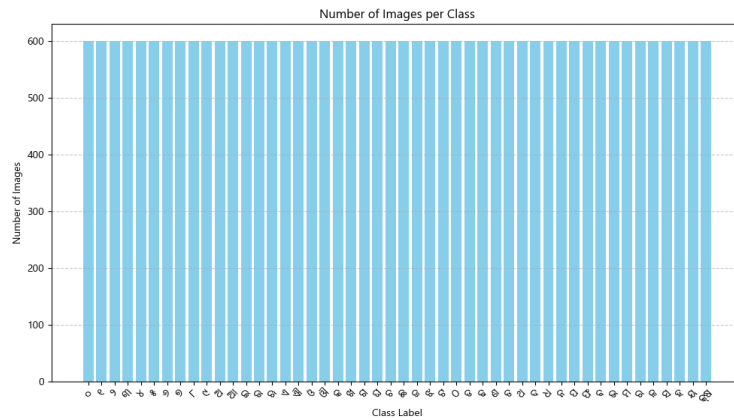


Figure 4.3 Distribution of images in train dataset (after augmentation)

For the test dataset, with 40% split, each of the 48 class have balanced 40 images.

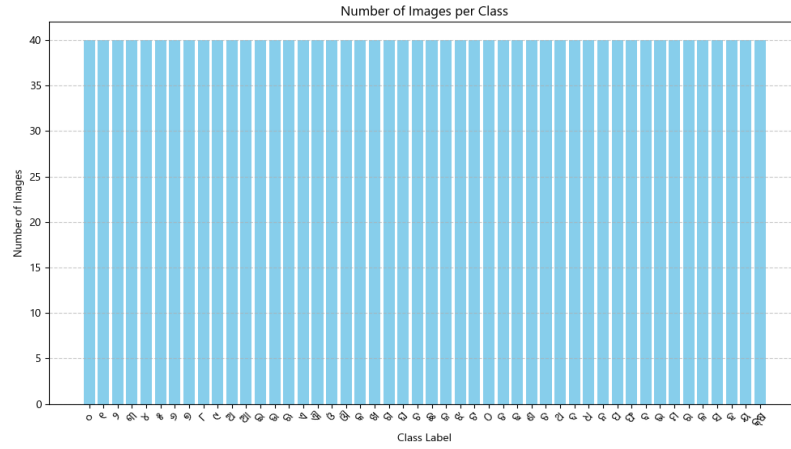


Figure 4.4 Distribution of images in test dataset

We collected the images in the dataset in various occasions, with different participants, to induce a degree of variability which is further enhanced by the augmentation.

4.2. Training Analysis

The Table 2, presents the observations recorded after training the model with different sets of features extracted using various feature extraction techniques, for the OSL dataset.

Table 2: Training observations

Models	Training accuracy	Hinge loss
ResNet50 + SVM	99.86%	0.023
Hybrid + SVM	83.17%	0.6694
EfficientNetB0 + SVM	99.61%	0.0067

As we can observe, the hybrid + SVM model, after extracting a greater number of features than the other models, still lacks in training accuracy. This may be due to the increasing complexity with lots of features.

4.3. Result Comparison and Analysis

We reviewed and utilized recent works from standard sources to compare with our observations. To justify the performance of our model, we used performance metrics such as accuracy, precision, and recall for comparison.

4.3.1. Performance Metrics

The classification report is one of the most commonly used tools for analyzing the performance of multiclass classification models. It is based on four parameters. Those are True Positive (TP) means cases where the model correctly predicts the

actual class, True Negative (TN), cases where the model correctly predicts the negative class, False Positive (FP) symbolizes cases where the model incorrectly predicts the actual class and False Negative (FN) i.e. cases where the model fails to predict the actual class. Using these values, the classification report provides four main evaluation metrics.

- Accuracy measures the overall correctness of the model.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \quad (19)$$

- Precision shows how many of the predicted positive cases are correct.

$$\text{Precision} = \frac{TP}{TP+FP} \quad (20)$$

- Recall (sensitivity) measures how many actual positive cases were correctly identified out of total cases.

$$\text{Recall} = \frac{TP}{TP+FN} \quad (21)$$

- F1 score is harmonic mean of precision and recall, providing a balance between them.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (22)$$

A higher value of accuracy, precision, recall, and F1 score indicates better model performance. High precision means that most of the positive predictions made by the model are correct, while high recall indicates that the model is successfully identifying most of the actual positive cases. On the other hand, a low precision value suggests that the model is producing many false positives, and a low recall value implies it is missing many true positives. The F1 score, which combines both precision and recall, is particularly important when dealing with imbalanced datasets, as it provides a balanced view of the model's performance by penalizing extreme values in either metric.

The confusion matrix is another widely used method for representing classification results, especially in a visual format. It displays the actual class labels along the vertical axis and the predicted class labels along the horizontal axis. To enhance readability and interpretation, the matrix is often visualized as a heatmap. In this representation, the diagonal elements indicate correct predictions. A higher concentration of values along the diagonal, often shown with deeper or darker colors, suggests better model performance, as it reflects that the predicted classes closely match the actual classes [35].

Other metrics includes Cohen's Kappa Score, Matthews Correlation Coefficient (MCC) and Hamming Loss. These metrics can also be used to analyses the result of the project.

- Cohen's Kappa Score measures the agreement between the predicted and actual

classifications, taking into account the possibility of agreement occurring by chance. [36]

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (23)$$

P_o = observed agreement

P_e = expected agreement by chance

- MCC is a balanced measure that takes into account true and false positives and negatives. It is especially useful for evaluating binary and multiclass classification problems, even when the class distribution is imbalanced. [37]

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}} \quad (24)$$

- Hamming Loss calculates the fraction of labels that are incorrectly predicted. It is the average of the number of incorrect labels to the total number of labels. [38]

$$Hamming Loss = \frac{1}{n} \sum_{i=1}^n 1(y_i \neq \hat{y}_i) \quad (25)$$

Where 1 is the indicator function, returning 1 if $(y_i \neq \hat{y}_i)$, and 0 otherwise.

Cohen's Kappa Score, Matthews Correlation Coefficient (MCC), and Hamming Loss are valuable metrics for evaluating classification performance, especially in cases involving class imbalance or multilabel outputs. A Cohen's Kappa Score of 1 indicates perfect agreement between predicted and actual labels, 0 reflects agreement equal to chance, and negative values suggest worse-than-chance performance. MCC provides a more balanced measure by considering all four elements of the confusion matrix (TP, TN, FP, FN), with values ranging from -1 to 1—where 1 indicates perfect prediction, 0 denotes random guessing, and -1 represents total disagreement. On the other hand, Hamming Loss measures the fraction of incorrect predictions; a lower value indicates better performance, with 0 meaning that all predictions are correct. These metrics offer deeper insights into model behaviour beyond simple accuracy.

We compared the result of our model with baseline model on ASL dataset. The comparison outcome of the proposed model shown in Table 3.

Table 3: ASL Result Comparison

Model Used	Dataset ASL Accuracy (weighted)
SVM (Polynomial Kernel) [39]	92.12%
EfficientNet-B0 [40]	98.96%
ResNet50 [41]	93%
ResNet50 + SVM	95.83%
Hybrid + SVM	95.83%
EfficientNetB0 + SVM	98.33%

In this section we discussed about other approaches we worked on during our project work and the outcomes we found. For result analysis, we used conventional methods commonly applied in multiclass image classification tasks. These included metrics such as accuracy, precision, recall, and F1-score. To visually represent the results, we used a confusion matrix heatmap showing the comparison between actual and predicted classes. Since our model outputs class labels directly for the test data, we could not use metrics like ROC-AUC. However, we employed other suitable metrics that work with label predictions, such as Cohen’s Kappa Score, Matthews Correlation Coefficient (MCC), Hamming Loss, and class-wise support. Additionally, we manually predicted each image in our test dataset to further evaluate the model’s performance. In the following sections, we compare the performance of our model with recent well-known research works and also evaluate how our model performs on the OSL dataset versus the existing ISL dataset. Based on these comparisons, we conclude with the best-performing model.

Table 4: OSL Result comparison

Models	OSL dataset						
	Cohen's Kappa Score	Matthews Correlation Coefficient	Hamming loss	precision	recall	F1 score	Accuracy
ResNet50 + SVM	0.9952	0.9952	0.0047	0.9957	0.9953	0.9954	99.53%
Hybrid + SVM	0.8505	0.8517	0.1463	0.8884	0.8536	0.8491	85.36%
EfficientNetB0 + SVM	0.9968	0.9968	0.00312	0.9970	0.9969	0.9969	99.69%

In the project work, we experimented with various machine learning and deep learning approaches for image classification. We used ResNet50 and a hybrid of EfficientNetB0 and ResNet50 as feature extractors. We also worked with fully connected neural network (FNN) layers as classifiers. However, the FNN layers were constrained by system capabilities and performed slowly on large and complex image datasets. Therefore, we preferred using the SVM classifier instead. In our proposed architecture, we experimented with ResNet50, the hybrid feature extractor, and the EfficientNetB0 model for feature extraction. The EfficientNetB0 feature extractor generated 1280 features, making it a lightweight model with good performance. ResNet50 extracted 2048 features, and the hybrid approach combined both, resulting in 3328 merged features.

After implementing the models on an existing American Sign Language dataset from the internet (as shown in Table 3), we obtained the highest accuracy of 98.33% using the EfficientNetB0 + SVM model, compared to 95.83% for both ResNet50 and the hybrid feature extractor. Thus, we chose the EfficientNetB0 + SVM model for further work. When we applied these three models to our own Odia Sign Language dataset (as shown in Table 4), we achieved an accuracy of 99.69% using EfficientNetB0 + SVM, 99.53% using ResNet50 + SVM, and 85.36% for the hybrid + SVM model. In individual image prediction on the test dataset (1920 images), EfficientNetB0 + SVM correctly predicted 1916 images (99.79%). However, ResNet50 + SVM only predicted 1893 images (98.59%) correctly.

Hence, due to its fast and smooth performance and higher accuracy compared to other approaches on the ASL dataset, we chose the EfficientNetB0 + SVM model. Given its strong performance on our OSL dataset and its ability to accurately predict unseen images, we propose the EfficientNetB0 + SVM model as the solution to our problem statement until further work is carried out on this topic.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion.

The application of deep learning, particularly Convolutional Neural Networks (CNNs), in Odia Sign Language detection presents a transformative step toward inclusive communication. By moving beyond traditional methods and focusing on the recognition of Odia letters individually through feature extraction and key point analysis, the proposed system enhances both accuracy and usability. Although the current accuracy is satisfactory, there is always room for improvement through the inclusion of more diverse datasets, varied working environments, and further optimization for real-time deployment. This approach not only simplifies the translation process but also enables smoother, more natural interactions between the deaf or hard-of-hearing community and non-signing individuals.

5.2 Future Scope

To enhance Odia Sign Language detection, key future directions include recognizing dynamic gestures using video sequences, expanding the dataset with diverse conditions (lighting, backgrounds, skin tones), and developing real-time mobile or web applications for live translation. Integrating speech/text translation, deploying lightweight models on edge devices, and supporting sentence formation using sign language grammar can significantly improve usability. A cross-language modular framework, AR-based real-time overlays, and active collaboration with the deaf community are also crucial for practical, inclusive, and scalable system development.

REFERENCES

- [1] J. P. Bonet, "Deaf History Europe," in *the first book on the subject of manual alphabetic signs for the deaf.*, 1602.
- [2] "Deaf History, Europe," [Online]. Available: <https://www.deafhistory.eu/index.php/component/zoo/item/1755#:~:text=Abb%C3%A9%20Charles%20Michel%20de%20l%27%C3%89p%C3%A9%20of%20Paris%20founded,system%20of%20conventional%20gestures%2C%20hand%20signs%2C%20and%20finge r spelling..> [Accessed 22 april 2025].
- [3] "The History and Evolution of American Sign Language: A Journey of Recognition and Empowerment," Heritage Interpreting, 7 april 2023. [Online]. Available: <https://heritageinterpreting.com/the-history-and-evolution-of-american-sign-language-a-journey-of-recognition-and-empowerment/>. [Accessed 22 april 2025].
- [4] "Ali Yavar Jung National Institute of Speech and Hearing Disabilities (Divyangjan)," [Online]. Available: <https://ayjnishd.nic.in/about-the-institute-2/>. [Accessed 22 april 2025].
- [5] "Basic indian sign language," ncert, [Online]. Available: <https://ciet.ncert.gov.in/activity/biSL>. [Accessed 22 april 2025].
- [6] S. S. & S. F. Sunila Maharjan, "Sign Language Detection and Translation Using Smart Glove," in *Springer*, Singapore, 2024.
- [7] R. K. Pranav Katarya, "Effi-CNN: real-time vision-based system for interpretation of sign language using CNN and transfer learning," *Multimedia tools and applications*, vol. 84, pp. 3137-3159, 2025.
- [8] S. R. Z. Hanan A. Taher, "A Critical Study of Recent Deep Learning-Based Continuous Sign Language Recognition," *The review of socionetwork strategies*, vol. 19, pp. 131-161, 2025.
- [9] B. A. A. S. A. A. I. M. & A. E. Alsharif, "Deep Learning Technology to Recognize American Sign Language Alphabet," *Machine Learning and AI for sensors*, vol. 23, no. 18, p. 7970, 2023.
- [10] V. S. S. M. N. K. B. P. Sukhavasi, "A Hybrid CNN-LSTM Architecture for Improved Sign Language Recognition," in *Springer*, 2025.
- [11] M. S. P. & F. G. F. Papatsimouli, "A Survey of Advancements in Real-Time Sign Language Translators: Integration with IoT Technology," *Technology Advances on IoT Learning and Teaching*, vol. 11, no. 4, p. 83, 2023.
- [12] A. C. A. Chakravarty, "Camera-Based ASL Alphabet Recognition Using Hand Landmark Features," in *Springer*, 2025.
- [13] J. X. Zhang Y, "Recent Advances on Deep Learning for Sign Language Recognition," *Computer Modeling in Engineering & Sciences*, vol. 139, no. 3, pp. 2399-2450, 2024.
- [14] Y. I. Rokade, "Indian Sign language recognition system," *researchgate*, 2017.
- [15] M. Taskiran, "A real-time system for recognition of american sign language by using deep learning," *Research gate*, 2018.
- [16] S. M. Ali, "Comparative Analysis of YOLOv3, YOLOv4 and YOLOv5 for Sign Language Detection," *IJAJRILE*, 2021.
- [17] D. Kothadiya, "Deepsign: Sign Language Detection and Recognition Using Deep Learning," *MDPI electronics*, 2022.
- [18] S. Mohishn, "American Sign Language Recognition Based on Transfer Learning Algorithm," *ijisae*, 2023.

- [19] B. Saini, "A comparative analysis of Indian sign language recognition using deep learning models," *Academic*, 2023.
- [20] H. Orovwode, "Development of a Sign Language Recognition System using machine learning," 2023.
- [21] J. Rambhia, "Real Time Indian Sign Language Recognition using Deep LSTM network," *IRJASET*, 2023.
- [22] S. Misal, "Sign language detection using machine learning," *IJISAR*.
- [23] "Tensorflow," 7 june 2024. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image_dataset_from_directory. [Accessed 21 april 2025].
- [24] L. Q. Tan M, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," 2019.
- [25] M. H. A. Z. M. Z. A. & C. L. Sandler, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *Computer Vision and Pattern Recognition*, pp. 4510-4520, 2018.
- [26] P. Z. B. & L. Q. V. Ramachandran, "wish: A Self-Gated Activation Function.," *Neural and Evolutionary Computing*, 2017.
- [27] B. K. A. L. A. O. A. T. A. Zhou, "Learning Deep Features for Discriminative Localization," *Computer Vision and Pattern Recognition*, 2015.
- [28] H. & H. A. Belyadi, "Machine learning workflows and types," *Machine Learning Guide for Oil and Gas Using Python*, vol. 2021, pp. 97-123.
- [29] Q. C. C. Z. Y. Liu, "Feature selection for support vector machines with RBF kernel," *Artificial Intelligence review*, vol. 36, pp. 99-115, 2011.
- [30] "svc," Scikit Learn, [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>. [Accessed 21 april 2025].
- [31] "NuSVC," Scikit-learn, [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.NuSVC.html>. [Accessed 21 april 2025].
- [32] C. V. V. Cortes, "Support-vector networks," vol. 20, pp. 273-297, 1995.
- [33] O. Fahey, "kaggle," 2022. [Online]. Available: <https://www.kaggle.com/datasets/lexset/synthetic-asl-alphabet>. [Accessed 3 may 2025].
- [34] "albumentation," [Online]. Available: <https://pypi.org/project/albumentations/>. [Accessed 21 april 2025].
- [35] M. & L. G. Sokolova, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427-437, 2009.
- [36] J. Cohen, "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement.*, vol. 20, no. 1, pp. 37-46, 1960.
- [37] B. Matthews, "Comparison of the predicted and observed secondary structure of T4 phage lysozyme," *Biochimica et Biophysica Acta (BBA) - Protein Structure*, vol. 405, no. 2, pp. 442-451, 1975.
- [38] Q. Z. a. Z. Z. S. Maggio, "Free-space optical backplane prototype for telecommunication equipment in the petabit/s range," *Bell Labs Technical Journal*, vol. 18, pp. 285-291, 2013.
- [39] J. P. M. Rokade Y I, "Indian Sign Language Recognition System," 2017.
- [40] S. D. G. P. A. M. A. k. P. Polaiya P, "Sign Language Detection Using Deep Learning," p. 6, 2023.
- [41] S. B. W. M. A. K. I. B. F. Z. S. R. M. Moshin S, "American Sign Language Recognition Based on Transfer," *International Journal of INTELLIGENT SYSTEMS AND APPLICATIONS IN ENGINEERING*, 2023.

